Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Jayden Pajanel	jaypaj@udel.edu
Friend (optional)		

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea and a paragraph describing how that paper supports your design concept.

        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider creating an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the ChemPy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages from uploaded images.
    - **Mechanical Engineering** majors may consider a project that somehow uses both the NumPy and SciPy Python libraries.

I want to create a story driven game where your choices affect the outcome. I've been playing videogames my entire life and would think that it would be cool if I created a game of my own.

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

There are plenty of tools to help create and/or spread story games. Websites like twine to create story games using code, or itch.io that hosts and distributes story games.

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

The purpose of this website is to be an interactive browser based game where the decisions you make change the outcome of the story. It will be used as a form of entertainment.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

The ultimate goal is by taking user decisions at an input, the input will be processed and will go through all the different outcomes that can happen, and will output the matching input.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

Enter their name and choose choices (Ex: a, b, c, or d)

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

It would just be a bunch of if else statements. If you choose different answers, you will get different responses.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

They will get an output in accordance with their input.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

Player, Location of the story, and the choices they make.

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

Need: Names, Health, Location *Ignore: Character traits, physical properties*

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

I believe the most difficult thing about the project would be the creativity to think of a good story game

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

I will ask LLM's to guide me on what my next steps should be for my code, and if there are better alternatives than what I am doing. Instead of asking for answers, I will ask for guidance.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

My ultimate goal overall is to just create a working project that I am proud of and also get a good grade on it.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

I want to build a choice-driven story game in Python for a web project. What are 3 core mechanics or features that make a text based story game fun to play, and what are 2 common design problems beginners run into when making story branches?

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

Give each decision weight behind it, and will change the course of the whole story. They also said be careful with your decisions because after 10 decisions, it creates 1024 different outcomes.

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

What is a simpler way to have different decisions without having a bunch of distinct decisions?

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

No, my idea on my game is around the same as when I started.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

I will try and use an LLM or the CISC textbook.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

I will probably ask a friend or go to office hours.

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

The only AI used was the part 5 research above.

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.

ChatGPT drafter conversation:

I want to build a choice-driven story game in Python for a web project. What are 3 core mechanics or features that make a text based story game fun to play, and what are 2 common design problems beginners run into when making story branches?
For a choice-driven story game, the most important thing is making players feel like their decisions matter. You don't need complicated technology to create an engaging experience. Here are three core mechanics and two common design problems to consider.

3 core mechanics that make a story game fun
1. Meaningful choices and consequences

Give players two or more choices at important moments.

Make choices affect what happens later in the story.

For example, helping a stranger might earn their trust, while ignoring them could make them an enemy later.

Ideally, choices should lead to different endings.

2. Branching storylines and multiple endings

Let players experience different parts of the story depending on their decisions.

Create several possible paths through the story.

Include at least 2–3 different endings, such as a good ending, bad ending, and secret ending.

Players may want to replay the game to discover what happens when they make different decisions.

3. Player stats or inventory

Track simple information about the player as they progress.

Examples include health, money, reputation, collected items, or trust.

Use these values to unlock choices or change story outcomes.

For example, a player who has a key in their inventory can unlock a special door.

2 common design problems with story branches
1. Branching complexity (too many story paths)

Every additional choice can create more possible story combinations. This can quickly become overwhelming.

For example:

First choice: 2 possible paths.

Second choice: 2 paths from each previous path.

Third choice: 2 paths from each previous path.

This creates 8 possible paths after only three decisions.

Simpler solution: Use a branching-and-rejoining structure. Let choices create different short experiences, but bring the story back together at certain major events.

2. Choices that don't actually matter

A common beginner mistake is giving players choices that appear important but lead to exactly the same outcome.

For example:

Choice A: Help the character.

Choice B: Ignore the character.

Both choices lead to the exact same dialogue and ending.

This makes players feel like their decisions are meaningless.

Simpler solution: Make choices affect at least one noticeable thing, such as dialogue, an inventory item, a character relationship, or the ending.

My recommendation for your Drafter project
Since this is an introductory Python web application, I'd recommend starting with:

Feature

Implementation

Story choices

Buttons for each decision

Branching

Track the current story scene

Player stats

Python variables or a dictionary

Inventory

A Python list

Multiple endings

Conditional statements (if/else)

Replay

A restart button

You could represent each story scene using a Python dictionary containing its description, choices, and the next scene for each choice.

One important scope recommendation: Avoid trying to build an enormous open-world story. A game with 8–12 scenes, 3 meaningful decisions, and 2–3 endings can be very enjoyable while remaining manageable for a CS1 project.

How this connects to your project worksheet

Your uploaded project worksheet asks you to identify the problem, define the input-process-output model, identify important data, and discuss technical challenges.

For your game, those could look like this:

Input: Player choices made using buttons.

Processing: Python evaluates choices and updates story progress, stats, and inventory.

Output: New story text, available choices, and eventually an ending.

Core data: Story scenes, player stats, choices, and inventory.

Biggest technical challenge: Managing branches without making the story too complicated.

This would also make a strong starting point for Part 5 (Research & Brainstorming Conversation) of your worksheet.

What is a simpler way to have different decisions without having a bunch of distinct decisions?
A simpler approach is to use small choices that change the story slightly, but eventually lead back to the same main storyline. This is called a branch-and-rejoin structure.

Instead of creating completely separate stories for every decision, you let players make choices that affect details along the way.

Example: A mystery adventure
Scene 1: You enter a mysterious house.

You see a locked door and a staircase.

Choice A

Try the locked door.

Choice B

Walk upstairs.

Door path

Find a flashlight.

Staircase path

Find a mysterious note.

Same next scene

You hear a strange noise coming from the basement.

Both decisions lead to the same next major scene, but the player has a different item or piece of information.

3 simple ways to make choices matter
Change dialogue: Characters respond differently depending on what the player chose.

Change inventory: Players collect different items that might help them later.

Change player stats: Choices increase or decrease something like trust, courage, or health.

How to program this in Drafter
You could use just a few Python variables:

```
player = {
    "flashlight": False,
    "has_note": False,
    "trust": 0
}
```

When a player makes a choice, update the appropriate variable. Later, use if statements to check those variables and change the story.

My recommendation: Build one main storyline with 2–3 decision points. Each decision changes a small detail, but the story eventually rejoins. Then use the player's accumulated choices or stats to determine which of 2–3 endings they receive.

This gives players a sense of freedom without requiring you to write dozens of different story paths.